

Distributed Algorithms - Project 2

João Carlos Antunes Leitão

NOVA Laboratory for Computer Science and Informatics (NOVA LINCS)

and

Departamento de Informática

Faculdade de Ciências e Tecnologia

Universidade NOVA de Lisboa

V 0.1

April 23, 2026

1 Overview

This document presents the specification of the second Distributed Algorithms project for the 2025/26 edition. The two projects are fully independent. The goal of this project is to implement and experimentally compare two leader-based agreement protocols — Multi-Paxos [3, 6] and Raft [4] — as the protocol for a replicated key-value store managed using a state machine replication (SMR) abstraction [5]. The key-value store application is provided to students; the central challenge is the correct and efficient implementation of both agreement protocols and of the state machine replication (SMR) layer that coordinates them.

Both protocols are foundational to modern distributed systems. Raft is the agreement protocol powering *etcd*, the coordination backbone of Kubernetes, and is also the foundation of CockroachDB and TiKV; Multi-Paxos and its variants are employed in fundamental services and abstractions for cloud computing systems, such as Google Chubby, Apache ZooKeeper, and Google Spanner. Despite sharing the same fundamental safety guarantees — agreement on a totally ordered log of commands, even in the presence of minority failures — the two protocols differ considerably in structure and design philosophy. Multi-Paxos amortises the cost of leader establishment across many slots by running Phase 1 only once per leader term; thereafter, only Phase 2 is required per slot, reducing message complexity in the common case. Raft, designed explicitly for understandability, tightly integrates leader election with log replication and enforces a strict log-matching invariant, making it arguably easier to reason about but structurally more prescriptive. Understanding how these design differences translate into observable performance characteristics is the central experimental question of this project.

Students will implement both protocols using the Babel framework [2], integrate them with an SMR layer, and conduct an experimental evaluation comparing their throughput and latency under a client workload generated by YCSB [1].

In the following, the target protocol architecture and interfaces are presented (Section 2). The programming environment is briefly discussed next (Section 3). The document concludes with operational aspects and evaluation criteria (Section 4).

2 Solution Architecture

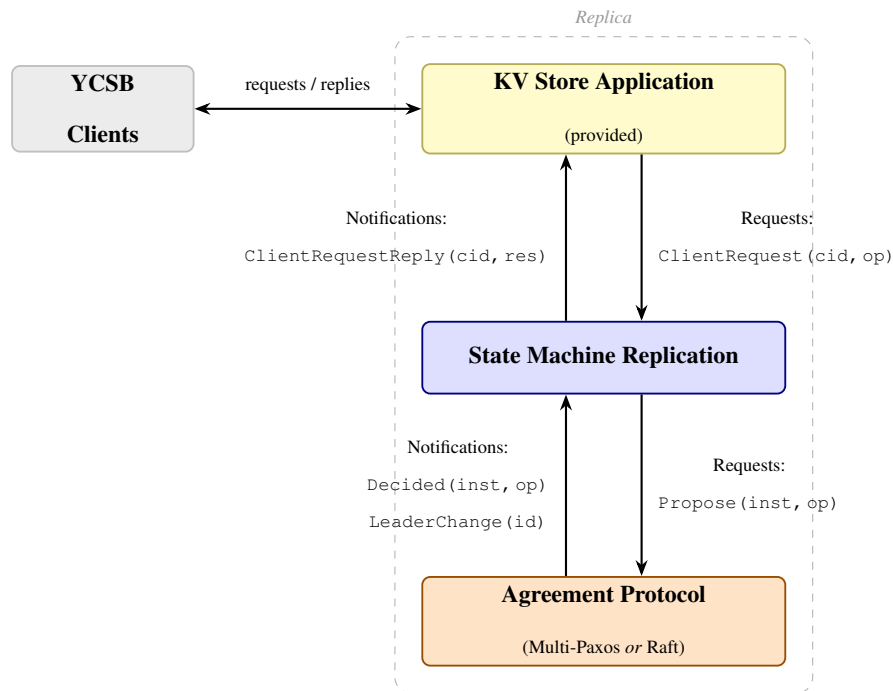


Figure 1: Architecture of a system replica and client interactions. The SMR and Agreement protocols additionally exchange messages with their counterpart instances on other replicas over the network.

Figure 1 illustrates the protocol stack that must be deployed on each system replica for this project¹. The figure also describes the interactions between the layers, from which the **interfaces** of the protocols can be directly derived. These interfaces must be respected in your implementations; the Java project that you will receive already provides the Babel events that materialise them.

Both the SMR protocol and the agreement protocol exchange messages with their respective counterparts on other replicas over the network. These network messages are internal to each protocol and are not prescribed; students should design them following the specifications of the protocols being implemented.

A critical design requirement is that the SMR protocol must be **as agnostic as possible** to the underlying agreement protocol. Swapping Multi-Paxos for Raft (or vice versa) should require no modification to the SMR layer beyond passing a configuration parameter that selects the agreement protocol. Both Multi-Paxos and Raft notify the SMR layer of leadership changes via the `LeaderChange` notification, which allows the SMR layer to handle leader-forwarding uniformly regardless of the agreement protocol in use (i.e., forwarding client requests to the correct replica).

Below, each protocol component is described in detail.

2.1 State Machine Replication

The SMR protocol is the coordinator of the replica. It receives operations issued by the application layer on behalf of clients, submits them to the agreement protocol for ordering, and ensures that decided operations are executed by the application in the correct sequence.

Concretely, the SMR protocol is responsible for the following:

¹For evaluation purposes, this layering is *mandatory*.

Operation ordering: Upon receiving a `ClientRequest` from the application, the SMR protocol submits the operation to the agreement protocol via a `Propose` request targeting the next available instance. When the agreement protocol delivers a `Decided` notification for instance k , the SMR protocol must execute operations strictly in instance order: if instance k has been decided but instance $k - 1$ has not yet, execution of instance k must be deferred until instance $k - 1$ is also decided and executed.

Leader forwarding: Both Multi-Paxos and Raft require that only the current leader proposes new operations. Replicas that receive a client operation but are not the leader must therefore forward that operation to the current leader. The SMR protocol tracks the identity of the current leader via `LeaderChange` notifications from the agreement protocol. Any operations that had been submitted but not yet decided when a leadership change occurs must be resubmitted to the new leader.

Connection management: The SMR protocol manages the TCP connections to the other replicas, which is necessary to forward operations to the leader and to receive forwarded operations from other replicas.

The system operates with a fixed, pre-configured set of replicas. No dynamic membership changes (replicas joining or leaving) are required for the base implementation. Students who additionally implement dynamic membership with state transfer earn bonus credit (see Section 4).

2.2 Agreement Protocol

Students must implement both of the following agreement protocols. In both cases, avoid optimisations such as pipelining multiple concurrent instances, as these require significant additional engineering effort and are not required for this project.

Multi-Paxos: Multi-Paxos extends single-decree Paxos [3] to support an ordered sequence of decisions (a replicated log). The key optimisation is that a replica acting as leader runs Phase 1 (Prepare) only once, obtaining a majority of `PrepareOk` replies to a sequence number that covers all future instances. Thereafter, the leader executes only Phase 2 (Accept) for each new operation slot, reducing message complexity to 1 round-trip-time (RTT) between replicas per decision in the common case. If the leader fails or becomes unreachable and a new replica initiates Phase 1 with a higher sequence number, it must also recover any potentially decided values from the slots that the previous leader was working on or that were already decided (for instance in periods of asynchrony or network partitions), following the standard Paxos safety argument. The protocol notifies the SMR layer of each leadership change via the `LeaderChange` notification. Students should implement Multi-Paxos following the specification covered in the lectures and reference in relevant papers [3, 6].

A practical implementation note: since the protocol handles multiple log slots simultaneously, it is advisable to represent the per-slot Paxos state as a Java object, and maintain a map from instance number to slot state. When a message arrives for a given instance, the corresponding state object is retrieved from the map and updated. All protocol messages must be tagged with the instance number they refer to.

Raft: Raft [4] is an agreement protocol for managing a replicated log, designed with a strong emphasis on understandability. It structures the agreement problem around three sub-problems: leader election, log replication, and safety. A leader is elected via randomised election timeouts and `RequestVote` RPCs; at most one leader can be elected per term. The elected leader accepts new entries and replicates them to followers via `AppendEntries` RPCs, which also serve as periodic heartbeats that suppress spurious elections. An entry is committed once the leader has successfully replicated it to a majority of servers. Once committed, the log-matching invariant guarantees that the entry is present in the logs of all future leaders. Like Multi-Paxos, the Raft implementation must notify the SMR layer of leadership changes via the `LeaderChange` notification. Students should implement Raft following the specification in [4]; Figure 2 of that paper provides a compact and self-contained summary of the complete protocol. Log compaction via snapshots is not required for the base implementation but earns bonus credit (see Section 4).

2.3 Application (Key-Value Store)

The application layer is provided to students as part of the Java project. It implements a simple replicated key-value store supporting two operations: `GET(key)`, which returns the value associated with a key, and `PUT(key, value)`, which inserts or updates a key-value pair. Both operations are executed with *strong consistency* semantics: even read operations are ordered through the agreement protocol, guaranteeing linearisability. The application handles all interactions with YCSB clients and uses the prescribed Babel interface to interact with the SMR layer.

2.4 Clients (YCSB)

Client load is injected using YCSB [1]. YCSB spawns multiple client threads, each emulating an independent client issuing operations to the key-value store. The workload should use a 50% write / 50% read distribution. YCSB reports both throughput (operations per second) and average operation latency (milliseconds), which are the primary metrics for evaluation.

If the desired number of client threads exceeds the capacity of a single physical machine, students may run multiple YCSB instances across different machines concurrently. In that case, throughput should be reported as the **sum** across all YCSB instances, and latency as the **average** across all YCSB instances.

2.5 Experimental Evaluation

Students will conduct an experimental evaluation comparing the throughput and latency of a system driven by Multi-Paxos against one driven by Raft. Experiments are conducted on the DI/NOVA LINC research cluster (see Section 4).

Setup

Each experiment uses a fixed number of replicas and varies the number of YCSB client threads to generate different load levels. The system must be deployed with a **minimum of 3 replicas**; students are strongly encouraged to additionally evaluate with **5 replicas** to expose any performance differences arising from the larger majority quorum.

Mandatory Experiments

Students must evaluate both Multi-Paxos and Raft under fault-free conditions, producing a **throughput-latency plot** in which each protocol is represented by a curve. Each data point on a curve corresponds to one experiment with a fixed number of client threads; points for the same protocol are connected sequentially as client count increases. Both curves should appear on the same axes, enabling direct comparison. In such a plot, curves that extend further to the right (higher throughput) and lower (lower latency) represent better-performing protocols. All axes must carry meaningful numerical labels, and the number of client threads for each data point must be discussed in the report text.

Encouraged Experiments

Students are strongly encouraged to conduct experiments that inject leader failures: abruptly terminating the current leader mid-run and observing the impact on throughput and latency as the surviving replicas elect a new leader. This is a natural dimension for comparing the two protocols, given the structural differences between the leader election mechanisms of Multi-Paxos and Raft.

Students may also consider additional dimensions, such as varying protocol-specific parameters (e.g., election timeout range, heartbeat interval) or studying the effect of scale. Every extra dimension explored will be reflected positively in the evaluation, but students are advised to prioritise the depth and trustworthiness of their mandatory experiments over breadth. As with Project 1, conducting a rigorous experimental evaluation requires significant planning and time; students are encouraged to start this work well before the deadline.

3 Programming Environment

Students will develop their project using the Java language (version 11 minimum). Development will use **Babel** [2], the framework developed at NOVA LINC² by Pedro Fouto, Pedro Ákos Costa, Nuno Preguiça, and João Leitão, already used in Project 1 and in the lab sessions.

The framework uses the Netty framework³ for inter-process communication through sockets, although it is designed to hide this from the programmer. The (potentially incomplete) javadoc of the framework can be found at: <https://asc.di.fct.unl.pt/~jleitao/babel/>.

4 Operational Aspects and Evaluation Criteria

Group Formation

Ideally, groups should remain the same as in Project 1. This is due to the (temporal) need of having the discussions of both projects colocated. This said, people can leave a group (and potentially do the second project alone), which means that groups can only be composed of subsets of the groups used to deliver the first project.

The DI and NOVA LINC² Research Cluster

The cluster is the same infrastructure used in Project 1. Technical specifications, reservation procedures, and password management documentation are available at: <https://cluster.di.fct.unl.pt>. All groups should use the credentials that were provided for the first project. If this is not possible you should contact the faculty. **Never use a weak password:** irresponsible use of shared infrastructure will be subject to disciplinary action per School regulations.

Evaluation Criteria

The project delivery consists of both source code and a written report formatted as a short academic paper, referred to hereafter simply as the *paper*.

The paper must be at most 10 pages long, including all figures, tables, and references. It should use a font size no larger than 10pt and be typeset in two-column format. It is highly recommended that you use L^AT_EX to write the paper; a template will be provided.

The paper must contain a clear description of the implementation and how it maps to the specification of the algorithms studied in lectures. The paper should describe all experimental work conducted: setup, parameters, results (in clear and readable plots), and a discussion of findings, including a comparison between the two protocols.

The project will be graded on a scale from 1 to 20 as follows:

Correctness and quality of the Multi-Paxos implementation: **5 values**

Correctness and quality of the Raft implementation: **5 values**

Correctness and quality of the SMR layer (including protocol agnosticism): **3 values**

Quality of the written report: **3 values**

Experimental evaluation: **4 values**

The following components are optional but earn bonus points that can compensate for losses elsewhere (the final grade is capped at 20):

Dynamic replica membership with state transfer — allowing a new replica to join the running system by receiving a snapshot of the current application state from an existing replica: **+2 bonus values**

²<http://nova-lincs.di.fct.unl.pt>

³<https://netty.io>

Raft log compaction via snapshots — implementing the snapshot mechanism described in Section 7 of [4], allowing the log to be truncated and replaced by a compact state snapshot: **+2 bonus values**

The quality and clarity of the paper will influence the final grade. Students with a poorly written paper may inadvertently be penalised in the evaluation of their protocol designs and correctness arguments.

Deadline and Delivery

Delivery of the project is due on **24 May 2026 at 23:59:59**.

The delivery shall be made by e-mail to `jc.leitao@fct.unl.pt` with the subject *DistAlg Project 2 Delivery*. The e-mail must contain two attachments: *i*) a zip file with the project source files — students must run `mvn clean` before zipping to avoid including generated build artefacts; the zip should include `src`, `pom.xml`, and any configuration files created; and *ii*) a PDF file with the written report.

Project deliveries that do not follow these guidelines precisely may not be evaluated, yielding an automatic grade of zero.

References

- [1] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154. ACM, 2010.
- [2] Pedro Fouto, Pedro Ákos Costa, Nuno Preguiça, and João Leitão. Babel: A framework for developing performant and dependable distributed protocols. In *41st International Symposium on Reliable Distributed Systems (SRDS)*, pages 146–155. IEEE, 2022.
- [3] Leslie Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):18–25, 2001.
- [4] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, 2014.
- [5] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, 1990.
- [6] Robbert van Renesse and Deniz Altinbuken. Paxos made moderately complex. *ACM Computing Surveys*, 47(3):1–36, 2015.